

Algorithmic Trading

Automated Trading Basics

Explained so simply a 10-year-old gets it

Teaching a computer to follow your proven rules — the pipeline, broker APIs, and why the kill-switch matters more than the strategy.

Stock Market Mastery Roadmap • Professional Level

Educational only — not investment, trading, or tax advice.

What if a **robot** could place your trades exactly by your rules — no fear, no greed, no missed alarms? That's **algorithmic trading**: teaching a computer to follow *your* proven plan. It's powerful — but it's a **helper that obeys your rules**, not a magic money machine. Your engineering skills make this a natural edge.



What algo trading IS (and is NOT)

- **IS — rule-based execution**: a computer runs the *exact* rules you'd follow by hand, faster and without emotion.
- **IS NOT** — a mysterious "**AI that prints money.**" That's a myth (and usually a scam). A bot only automates an edge **you already have**.

AUTOMATION AMPLIFIES YOUR EDGE OR YOUR BUG

A robot does *exactly* what you told it — **thousands of times, instantly**. If your rules are good, that's wonderful. If there's a **bug** (or the edge is fake), it **loses money faster and quieter** than you ever could by hand. Automation is a megaphone: it makes whatever you feed it **louder**.



The stack: how the pieces connect

An algo is a **pipeline** — each stage hands off to the next:

Stage	What it does
1. Data feed	live prices come in (the robot's eyes)
2. Strategy logic	your rules decide: is there a setup?
3. Signal	a clear "buy/sell" instruction is produced
4. Risk checks	guards verify: is this allowed? (size, limits)
5. Order routing	the order is sent to the broker's API
6. Logging	everything is recorded for review

Notice **risk checks come before the order** — that ordering is deliberate and vital.



Broker APIs & data (India)

- **Broker API** — a doorway that lets your code place orders (e.g., **Kite Connect** and similar).
- **REST** — you *ask* for data when you need it (like checking a webpage).
- **WebSocket** — data **streams to you live**, tick by tick (like a phone line kept open) — best for real-time.



Types of automation

- **Execution algos (TWAP / VWAP)** — quietly split a big order over time to get a fair average price.
- **Signal automation** — the bot spots your setup and fires the order end-to-end.
- **Alerts-to-orders** — a TradingView alert → **webhook** → **your server** → **broker API** — a popular retail pipeline you can build.



Latency: retail is NOT high-frequency trading

DON'T CHASE MICROSECONDS

Big HFT firms fight over **millionths of a second**. As a retail trader, you **can't and shouldn't** compete there. Your edge is a **good strategy run reliably** — focus on **correct risk checks, clean logging, and rock-solid reliability**, not speed. A dependable slow bot beats a fast flaky one.



Guardrails: safety is the FIRST thing you code

- **Kill-switch** — one button (or rule) that **instantly stops everything** and flattens positions.
- **Max-loss guard** — code that **halts trading** after a set daily loss — no exceptions.
- **Position / loss limits in code** — hard caps the bot **physically cannot** exceed.

- **SEBI guardrails** — follow the regulator's rules for retail algos; stay compliant.

THE RULE THAT SAVES ACCOUNTS

A **kill-switch** and **max-loss guard** matter **MORE** than the strategy logic itself. Code the **safety limits FIRST**, before the clever part. And **never leave an unsupervised bot running without hard risk limits** — a small bug at 2× speed with no brakes can do damage no human ever could. **Test that the kill-switch actually fires.**



Key ideas to remember

- **Automation amplifies your edge OR your bug** — test relentlessly, and **code the risk limits first**.
- **A kill-switch and max-loss guard are more important than the strategy logic.**
- **Reliability and monitoring beat cleverness** for retail algos — boring and dependable wins.

PRACTICAL EXERCISES

- 📐 **Diagram the full flow:** TradingView alert → webhook → Node/Express server → broker-API order — with a **risk-check layer** and a **kill-switch** drawn in.
- 📄 **Build a paper-trading bot that only logs intended orders** (no real execution) and **audit its decisions for a week**.
- 🚧 **Add hard limits in code:** max daily loss, max positions, and an **emergency square-off**.

REAL-WORLD APPLICATION

- 💻 **This aligns directly with your engineering skills.** A well-tested **webhook-to-broker pipeline** with strict risk guards can execute a **proven manual edge without emotion** — removing the human weak point. But the order matters: **prove the edge by hand first, then** automate it.

ADVANCED PITFALL: AUTOMATING AN UNPROVEN EDGE

Automating an **unproven** edge just **loses money faster and quieter**. The bot won't fix a bad strategy — it'll execute it flawlessly into the ground. **Prove it by hand and in forward-testing first.** And **never deploy a bot without a max-loss kill-switch that you've tested to actually fire.** Untested safety is no safety.

ONE-LINE SUMMARY

Algo trading = a **robot obeying your proven rules**, not magic AI. Build the **pipeline** (data → logic → signal → **risk checks** → order → logging) on a broker API, forget **HFT speed**, and above all **code safety first**: a tested **kill-switch** and **max-loss guard** matter more than the strategy. Automation **amplifies edge or bug** — so prove the edge by hand before you ever let a bot run it.